EMG Gesture Classification — Progress Report

Wearable Rehabilitation Exoskeleton Project — Aihik Basu

Repository (Private)

github.com/aihikbasu/-EXO-REHAB-PROJECT — access restricted to collaborators.

Summary

As discussed, the data files shared (f1.txt, f7.txt) were unlabeled test samples without activity timing, so they could not be used for training. Per your suggestion, this version uses a public, labeled EMG dataset instead: the UCI 'EMG Data for Gestures' dataset (raw multi-channel EMG from a Myo armband, with a gesture label on every sample, from real subjects). Data from 10 subjects (20 recording sessions) was used, covering the four target gestures: rest, flexion, extension, and grasp.

Methodology Note

Raw recordings were sliced into overlapping signal windows, with features extracted per window (zero crossings, slope sign changes, mean absolute value, waveform length, RMS, variance). An initial version of this pipeline split windows into train/test independently, which let near-identical overlapping windows leak across the split and produced an inflated, meaningless 100% accuracy. This was caught and corrected: windows are now split by gesture repetition, not individually, so all windows from a given repetition stay entirely on one side of the split. The results below reflect genuine performance on unseen repetitions.

Results

2,470 labeled windows across 4 balanced classes, from 10 subjects.
SVM (RBF kernel): 96.0% accuracy
Random Forest: 96.2% accuracy
Rest and flexion are classified almost perfectly. The main confusion is grasp vs. flexion, consistent with both gestures activating similar finger-flexor muscles.

Code: Feature Extraction (features.py)

Extracts six standard EMG features per channel from a raw signal window.

```
import numpy as np


def zero_crossings(window, threshold=1e-6):
    signs = np.sign(window)
    signs[np.abs(window) < threshold] = 0
    crossings = 0
    for i in range(1, len(signs)):
        if signs[i] != 0 and signs[i - 1] != 0 and signs[i] != signs[i - 1]:
            crossings += 1
    return crossings


def slope_sign_changes(window, threshold=1e-6):
    diffs = np.diff(window)
    count = 0
    for i in range(1, len(diffs)):
        if diffs[i] * diffs[i - 1] < 0 and abs(diffs[i] - diffs[i - 1]) > threshold:
            count += 1
    return count


def mean_absolute_value(window):
    return float(np.mean(np.abs(window)))


def waveform_length(window):
    return float(np.sum(np.abs(np.diff(window))))


def rms(window):
    return float(np.sqrt(np.mean(np.square(window))))


def variance(window):
    return float(np.var(window))


def extract_channel_features(window):
    window = np.asarray(window, dtype=float)
    return {
        "zc": zero_crossings(window),
        "ssc": slope_sign_changes(window),
        "mav": mean_absolute_value(window),
        "wl": waveform_length(window),
        "rms": rms(window),
        "var": variance(window),
    }


def extract_features(channels):
    """channels: list of raw sample arrays, one per EMG channel."""
    row = []
    for ch in channels:
        feats = extract_channel_features(ch)
        row.extend([feats["zc"], feats["ssc"], feats["mav"], feats["wl"], feats["rms"], feats["var"]])
    return row
```

Code: Dataset Construction (build_dataset.py)

Segments raw labeled recordings into overlapping windows, grouped by gesture repetition to prevent train/test leakage, and extracts features per window.

```
"""
Turns the raw UCI EMG gesture recordings into a windowed, labeled feature
dataset ready for training. Each file is a continuous recording with a
class label per sample; this script slices contiguous same-label runs into
fixed-size windows and extracts features per window using features.py.

Label mapping (from the UCI 'EMG Data for Gestures' dataset):
  0 - unmarked           -> dropped
  1 - hand at rest       -> rest
  2 - hand clenched fist -> grasp
  3 - wrist flexion      -> flexion
  4 - wrist extension    -> extension
  5 - radial deviation   -> dropped (not one of the four target gestures)
  6 - ulnar deviation    -> dropped
"""

import glob

import numpy as np
import pandas as pd

from features import extract_features

LABEL_MAP = {1: "rest", 2: "grasp", 3: "flexion", 4: "extension"}
WINDOW_SIZE = 200  # samples per window
STEP = 100  # 50% overlap

CHANNEL_COLS = [f"channel{i}" for i in range(1, 9)]


def load_subject_file(path):
    df = pd.read_csv(path, sep="\t")
    df.columns = [c.strip() for c in df.columns]
    return df


def windows_for_run(df_run, label, run_key):
    """Slide fixed windows over one contiguous same-label run.

    run_key is carried through to every window from this run so that, at
    train/test split time, overlapping windows from the same gesture
    repetition can be kept together instead of leaking across the split.
    """
    rows = []
    values = df_run[CHANNEL_COLS].values
    n = len(values)
    for start in range(0, n - WINDOW_SIZE + 1, STEP):
        window = values[start:start + WINDOW_SIZE]
        channels = [window[:, i] for i in range(window.shape[1])]
        feats = extract_features(channels)
        rows.append(feats + [label, run_key])
    return rows


def build_from_file(path):
    subject = path.split("/")[-1].replace(".txt", "")
    df = load_subject_file(path)
    df = df[df["class"].isin(LABEL_MAP.keys())].reset_index(drop=True)

    all_rows = []
    # group contiguous runs of the same class (gestures are performed in blocks)
    df["run_id"] = (df["class"] != df["class"].shift()).cumsum()
    for run_id, run_df in df.groupby("run_id"):
        label = LABEL_MAP[run_df["class"].iloc[0]]
        run_key = f"{subject}_run{run_id}"
```

```
        all_rows.extend(windows_for_run(run_df, label, run_key))
    return all_rows


def main():
    files = sorted(glob.glob("data/subject_*.txt"))
    print(f"Found {len(files)} subject files: {files}")

    columns = []
    for ch in range(1, 9):
        columns += [f"ch{ch}_zc", f"ch{ch}_ssc", f"ch{ch}_mav", f"ch{ch}_wl", f"ch{ch}_rms", f"ch{ch}_var"]
    columns.append("label")
    columns.append("run_id")

    all_rows = []
    for path in files:
        rows = build_from_file(path)
        print(f"  {path}: {len(rows)} windows")
        all_rows.extend(rows)

    out = pd.DataFrame(all_rows, columns=columns)
    out.to_csv("data/emg_gesture_dataset.csv", index=False)
    print(f"\nTotal windows: {len(out)}")
    print(out["label"].value_counts())


if __name__ == "__main__":
    main()
```

Code: Training and Evaluation (train.py)

Trains SVM and Random Forest classifiers with a grouped train/test split, and outputs accuracy, a full classification report, and confusion matrices.

```
import json

import matplotlib.pyplot as plt
import pandas as pd
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import ConfusionMatrixDisplay, accuracy_score, classification_report
from sklearn.model_selection import GroupShuffleSplit
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC


def load_data(path="data/emg_gesture_dataset.csv"):
    df = pd.read_csv(path)
    X = df.drop(columns=["label", "run_id"])
    y = df["label"]
    groups = df["run_id"]
    return X, y, groups


def train_and_evaluate(model, name, X_train, X_test, y_train, y_test, labels):
    model.fit(X_train, y_train)
    preds = model.predict(X_test)
    acc = accuracy_score(y_test, preds)

    print(f"\n=== {name} ===")
    print(f"Accuracy: {acc:.3f}")
    print(classification_report(y_test, preds, labels=labels))

    disp = ConfusionMatrixDisplay.from_predictions(
        y_test, preds, labels=labels, xticks_rotation=30,
    )
    disp.figure_.set_size_inches(6, 5)
    disp.figure_.tight_layout()
    disp.figure_.savefig(f"confusion_matrix_{name.lower().replace(' ', '_')}.png", dpi=150)
    plt.close(disp.figure_)

    return acc


def main():
    X, y, groups = load_data()
    labels = sorted(y.unique())

    # Split by gesture repetition (run), not by individual window -- windows
    # from the same run overlap heavily and would leak across train/test if
    # split independently, inflating accuracy artificially.
    splitter = GroupShuffleSplit(n_splits=1, test_size=0.25, random_state=42)
    train_idx, test_idx = next(splitter.split(X, y, groups=groups))
    X_train, X_test = X.iloc[train_idx], X.iloc[test_idx]
    y_train, y_test = y.iloc[train_idx], y.iloc[test_idx]

    scaler = StandardScaler()
    X_train_scaled = scaler.fit_transform(X_train)
    X_test_scaled = scaler.transform(X_test)

    results = {}

    svm = SVC(kernel="rbf", C=10, gamma="scale")
    results["svm"] = train_and_evaluate(svm, "SVM", X_train_scaled, X_test_scaled, y_train, y_test, labels)

    rf = RandomForestClassifier(n_estimators=200, random_state=42)
    results["random_forest"] = train_and_evaluate(rf, "Random Forest", X_train, X_test, y_train, y_test, labels)

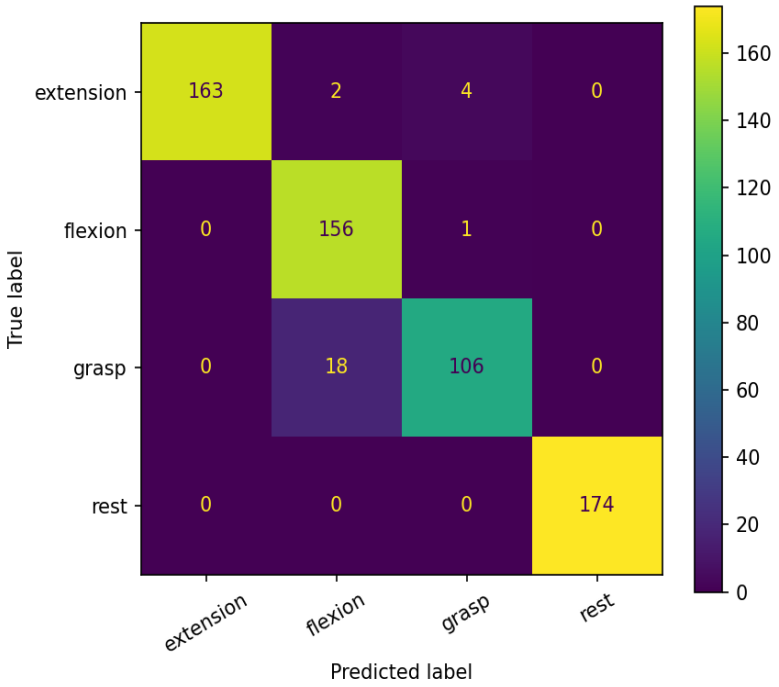
    with open("results.json", "w") as f:
        json.dump(results, f, indent=2)
```

```
    print("\nSummary:")
    for name, acc in results.items():
        print(f"  {name}: {acc:.3f}")

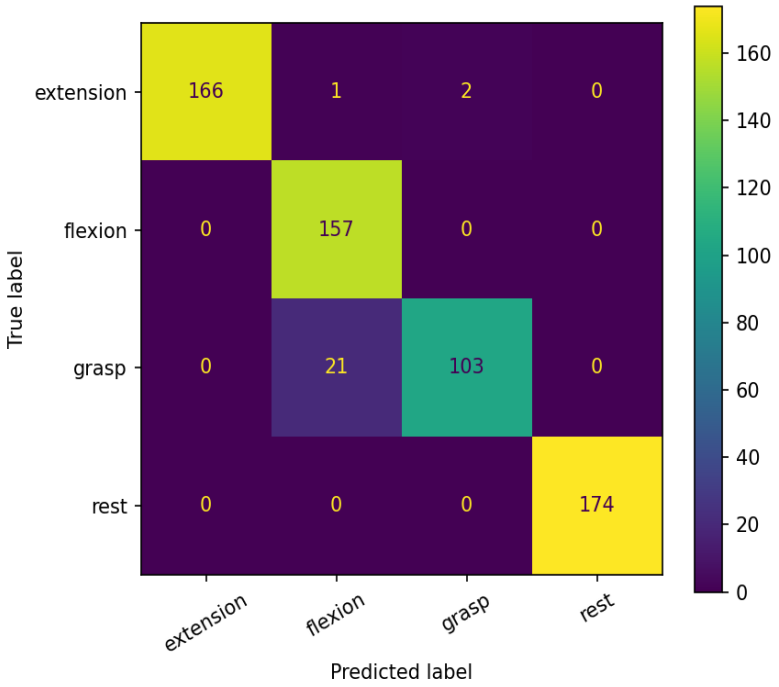

if __name__ == "__main__":
    main()
```

Confusion Matrices

SVM



Random Forest



Next Steps

Replace this public dataset with live data from the lab's own EMG sensors, once an activity-timing protocol is available for it; build a loader matching the lab sensors' actual file format; connect classifier output to the exoskeleton GUI's Run Trial page to directly drive the assistance strategy / actuator control.